# MOD -03 - JDK new Features

# MOD -03 - JDK new Features

LUMINAR
TECHNOLAB

# Optional class

1. Introduced in java 8
2. in java.util package
3. It is used to represent a value is present or absent.
4. Null checks are not required.
5. No more NullPointerException at run-time.
6. We can develop clean and neat application.
7. No more Boiler plate code

```java
package optionalpackage;

public class WithoutOptionalArrayDemo {
    public static void main(String[] args) {

        String[] arr = new String[10];
        //Getting the substring
        String sub = arr[6].substring(2, 5);
        //Displaying substring
        System.out.print(sub);

  }

}

```

Problems | Javadoc | Declaration | Console | Outline

<terminated> WithoutOptionalArrayDemo [Java Application] C:\Program Files\Java\jdk-11.0.13\bin\javaw.exe

```
Exception in thread "main" java.lang.NullPointerException
        at optionalpackage.WithoutOptionalArrayDemo.main(WithoutOptionalArrayDemo.java:8)
```

```java
package optionalpackage;

import java.util.Optional;

public class WithOptionalArrayDemo {
    public static void main(String[] args) {

            String[] arr = new String[10];
            Optional<String> isNull = Optional.ofNullable(arr[6]);
            if(isNull.isPresent()){
                //Getting the substring
                String sub = arr[6].substring(2, 5);
                //Displaying substring
                System.out.print(sub);
            }
            else{
               System.out.println("Cannot get the substring from an empty string");
            }

```

```java
        }

        arr[6] = "Welcome to Luminar";
        Optional<String> isNulll = Optional.ofNullable(arr[6]);
        if(isNulll.isPresent()){
            //Getting the substring
            String subs = arr[6].substring(2, 5);
            //Displaying substring
            System.out.print("Substring is: "+ subs);
        }
        else{
            System.out.println("Cannot get the substring from an empty string");
        }

    }
}
```

Problems | Javadoc | Declaration | Console | Outline

<terminated> WithOptionalArrayDemo [Java Application] C:\Program Files\Java\jdk-11.0.13\bin\javaw.exe

```
Cannot get the substring from an empty string
Substring is: lco
```

```java
package optionalpackage;

import java.util.Optional;

public class OptionalMore {

    public static void main(String[] args) {


        //Creating Optional object from a String
        Optional<String> ofc = Optional.of("Luminar Technolab");

        //Optional.empty() creates an empty Optional object
        Optional<String> nothing = Optional.empty();

```

```java

    //==========================================================================
      // isPresent() method: Checks whether the given Optional
      // Object is empty or not.
       System.out.println("=====isPresent====");
       if (ofc.isPresent()) {
          System.out.println("We are @ Luminar");
       }
       else {
          System.out.println("No class today");
       }
```

```java
    //==========================================================================

      //returns the value of Optional Object if it is no empty,
      //if the object is empty then it returns the default value


    //orElse() method
      System.out.println("=====orElse====");
      System.out.println(ofc.orElse("value ..."));
      System.out.println(nothing.orElse("value ..."));
```

```java
        //=================================================================================

        //Filter on Optional
          Optional<String> myName = Optional.of("BINI");
          System.out.println("=====filters====");
          System.out.println(myName.filter(g -> g.equals("bini"))); //case sensitive
          System.out.println(myName.filter(g -> g.equals("BINI")));
          System.out.println(myName.filter(g -> g.equalsIgnoreCase("bini")));
          System.out.println(nothing.filter(g -> g.equals("bini"))); //checking nothing object
    }

}
```

```
=====isPresent====
We are @ Luminar
=====orElse====
Luminar Technolab
value ...
=====filters====
Optional.empty
Optional[BINI]
Optional[BINI]
Optional.empty
```

# New Date/Time API

New date-time API is introduced in Java 8 to overcome the following drawbacks of old date-time API :

1. Not thread safe : Unlike old java.util.Date which is not thread safe
2. Less operations : In old API there are only few date operations but the new API provides us with many date operations.

Java 8 under the package java.time introduced a new date-time API, most important classes among them are :

1. Local : Simplified date-time API with no complexity of timezone handling.
2. Zoned : Specialized date-time API to deal with various timezones.

```java
package datetimepackage;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;

public class DateTimeDemo {
    public static void main(String[] args) {


        // the current date
        LocalDate date = LocalDate.now();
        System.out.println("the current date is "+date);

      // the current time
        LocalTime time = LocalTime.now();
        System.out.println("the current time is "+time);
```

```java

        // will give us the current time and date
        LocalDateTime current = LocalDateTime.now();
        System.out.println("current date and time : "+ current);

    // to print in a particular format
        DateTimeFormatter format =DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
        String formatedDateTime = current.format(format);
        System.out.println("in formatted manner "+formatedDateTime);

        // printing months days and seconds
        Month month = current.getMonth();
        int day = current.getDayOfMonth();
        int seconds = current.getSecond();
        System.out.println("Month : "+month+" day : "+day+" seconds : "+seconds);

    // to get the current zone
        ZonedDateTime currentZone = ZonedDateTime.now();
        System.out.println("the current zone is "+currentZone.getZone());

```

```java

      // getting time zone of specific place use withZoneSameInstant():
       //return a copy of this date-time with a different time-zone
         ZoneId india = ZoneId.of("Asia/Calcutta");
         ZonedDateTime indiaZone =currentZone.withZoneSameInstant(india);
         System.out.println("indian time zone is " +indiaZone);

    }

}

```

# Thank you . Happy Learning ☺